# Token Due-Diligence Score Workflow Overview

## Workflow — token-dd-score (Skill #24)

Step-by-step derivation of the **Token Due-Diligence Score** produced by `skills/token-duediligence/token-dd-score.py`. Skill description & CLI reference: `doc.md`.

> Every ramp in this document is **continuous** (smoothstep). Magnitudes that span several orders (liquidity, market cap) use a **log-spaced ramp**. Every rate coming from the API is expressed as a decimal fraction and multiplied by 100 exactly once. An absent field is **dropped together with its weight**, so a pillar score always reflects the signals that were actually measurable.

---

### 0. Pipeline at a glance

```
Step 1  token_overview            (1 call, all 8 frames)  21 CU
Step 2  token_security                                      30 CU
        ---------------------------- standard ends here (51 CU)
Step 3  holder distribution + holder-profile                60 CU
Step 4  top holders + token money flow                      40 CU
Step 5  liquidity OHLC (100 candles / 24h)                  30 CU
        ------------------------- advanced total 181 CU ---

Step 6  score 5 pillars  ->  weighted mean  ->  × gate
Step 7  rating band + verdict  ->  render (ui/json/telegram/plain)  ->
cache
```

`standard` = **Step 1 + Step 2 only**. `advanced` = all steps. Any endpoint that is Solana-only and the chain is not Solana is skipped and its CU is booked to `meta.cu_breakdown.refunded_rollback`.

#### Step 1 — Token overview (single call, 8 frames)

**Endpoint:** `/defi/token_overview?includes=money_flow` · **21 CU**

One call returns the full trade-dynamic grid. Frames consumed:

| frame | fields used |
|---|---|
| 1m, 5m, 15m, 30m, 1h, 2h, 4h, 24h | `vBuy*USD`, `vSell*USD`, `v*USD`, `uniqueWallet*`, `buy*`, `sell*`, `priceChange*Percent` |

Plus the token-level fields: `holder`, `v24hUSD`, `marketCap`, `fdv`, `liquidity`, `circulatingSupply`, `totalSupply`, `trade24h`, `uniqueWallet24h`, `price`, `priceChange{1m..24h}Percent`, `moneyFlow[]`.

**Money-flow normalisation** (used by Activity and Smart Money):

```
_mf_net_24h = moneyFlow[frame == "24h"].netUsd
              (fallback: volumeBuyUsd - volumeSellUsd)
```

`frames_used` is reported in `meta`; if no usable frame is present the response carries ⚠ `pipeline_gap — token_overview returned no usable frame data`.

---

### Step 2 — Token security

**Endpoint:** `/defi/token_security` · **30 CU**

#### Sub-signals and weights (Security pillar)

| sub-signal | weight | rule |
|---|---|---|
| `mint_authority` | 0.20 | Solana: `ownerAddress` null ⇒ renounced = 100, live ⇒ 0. EVM: `mintable == true` ⇒ 0 else 100 |
| `freeze_authority` | 0.18 | **Solana only** (`freezeAuthority` null ⇒ 100, live ⇒ 0). EVM ⇒ N/A |
| `metadata` | 0.08 | `mutableMetadata == false` ⇒ 100, true ⇒ 40 |
| `lp_lock` | 0.14 | `lpLocked == true` ⇒ 100 else 0 |
| `creator_dump` | 0.20 | see below |
| `not_freezable` | 0.05 | **Solana only** — freeze **and** mint both renounced ⇒ 100 |
| `not_lst` | 0.03 | **Solana only** — symbol not in the LST list ⇒ 100 |
| `not_fake` | 0.02 | symbol does not impersonate a |

| | | |
|---|---|---|
| | | canonical mint ⇒ 100 |

#### Creator holding × dump power (the "can they dump?" question)

The creator's holding is only dangerous if they also **retain the authority to transfer / exit**. That is combined multiplicatively:

```
creator_pct  = creatorPercentage × 100              # API gives 0-1
f_hold       = smoothstep(ramp(creator_pct, 5 %, 30 %))
n_auth       = (ownerAddress live) + (freezeAuthority live) +
(mutableMetadata)
f_power      = 0.6 + 0.4 × min(n_auth, 3) / 3

creator_dump = 100 × (1 - f_hold × f_power)
```

Reading the formula:

**Ramps table**

| sub-signal | ramp (lo → hi) | direction |
|---|---|---|
| `f_hold` | 5 % → 30 % | higher holding ⇒ larger penalty factor |

**Hard block:** `isHoneypot == true` ⇒ Security = 0 and the whole run is forced `final = 0`, rating `F`.

**Example.** Creator holds 12 %, mint renounced, freeze live, metadata mutable:

```
f_hold  = ramp_s(12, 5, 30) = smooth(0.2800) = 0.2800² × (3 - 0.5600) =
0.1913
n_auth  = 0 (mint) + 1 (freeze) + 1 (metadata) = 2
f_power = 0.6 + 0.4 × 2/3 = 0.8667
creator_dump = 100 × (1 - 0.1913 × 0.8667) = 100 × 0.8342 = 83.4 / 100
```

---

### Step 3 — Holder concentration & cohorts (advanced)

**Endpoints:** `/holder/v1/distribution?mode=top&top_n=N&include_list=true` (30 CU, Solana only) and `/token/v1/holder-profile` (30 CU, Solana only — returns the native cohort tags).

#### 3.1 One scale for the whole list

`percent_of_supply` is already 0–100 per row. The concentration figures are computed from **one sorted list with one scale**:

```
pcts = sorted([h.percent_of_supply for h in holders], reverse=True)
top1  = pcts[0]
top3  = Σ pcts[:3]
top10 = Σ pcts[:10]
```

`top5` is still derived and shown in the output for transparency, but it no longer feeds the holder pillar score.

#### 3.2 Concentration penalty ramps

`score = 100 × (1 - smoothstep(ramp(pct, lo, hi)))`

| sub-signal | weight | lo → hi | rationale |
|---|---|---|---|
| `conc_top1` | **0.23** | 10 % → 95 % | one wallet dumping is still the single biggest instant-rug vector |
| `conc_top3` | 0.15 | 20 % → 98 % | |
| `conc_top10` | 0.17 | 20 % → 100 % | "float" — the top-10 shell decides the exit |
| `holder_base` | **0.20** | log10 1.7 (≈50) → 3.7 (≈5 000) | healthy = many holders |
| `cohort_risk` | 0.10 | dev+sniper+bundler 5 % → 35 % | same idea as junk-token scoring |
| `sniper_density` | **0.15** | 5 → 20 sniper wallets | sniper **count** is itself a signal |

The holder pillar now gives a bit more credit to a real holder base and a bit more penalty power to sniper density, while dropping `top5` from the scored set.

#### 3.3 Cohorts

`holder-profile.tags[]` is a **list** of `{tag, holder_count, hold_amount, percent_of_supply, buy_volume_usd, sell_volume_usd, avg_buy_price, pnl}`. Native tags: `dev`, `bundler`, `sniper`, `insider`, `smart_trader`, `kol`. All six are reported in `holder_cohorts` (count + holding % + PnL); `dev`, `sniper`, `bundler` are the ones that feed `cohort_risk`, and `insider` is displayed as additional context.

**Worked example — advanced run of `BPxxfRC…jPCy`**

| sub-signal | input | ramp | sub-score | weight | points |
|---|---|---|---|---|---|
| conc_top1 | 75.00 % | 0.8600 | 14.0 | 0.23 | 3.22 |
| conc_top3 | 95.43 % | 0.9968 | 0.3 | 0.15 | 0.05 |

| | | | | | |
|---|---|---|---|---|---|
| conc_top10 | 96.40 % | 0.9941 | 0.6 | 0.17 | 0.10 |
| holder_base | 29 724 | 1.0000 | 100.0 | 0.20 | 20.00 |
| cohort_risk | dev 0.0 + sniper 0.0 + bundler 0.30 = 0.30 % | 0.0000 | 100.0 | 0.10 | 10.00 |
| sniper_density | 17 wallets | 0.8960 | 10.4 | 0.15 | 1.56 |
| **Holders** | | | | **1.00** | **≈ 34.9 / 100** |

`cohort_risk` ramp: `ramp_s(0.30, 5, 35) = 0` → no penalty. `sniper_density`: `ramp(17, 5, 20) = 0.8000` → `smooth = 0.8² × (3 - 1.6) = 0.8960` → `100 × (1 - 0.8960) = 10.4`.

Interpretation: 29.7k holders is genuinely healthy, but **top1 = 75 %** and **top10 = 96.4 %** are still crushing penalties, and `17` sniper wallets now matter much more because the sniper ramp is tighter. The pillar still scores low despite a strong holder base, which is the intended behaviour.

#### 3.4 Standard vs advanced for this pillar

In `standard` mode there is **no** `distribution` and **no** `holder-profile`, so `conc_top1/3/10` come from `token_security.top10HolderPercent` only (top10 is available, top1/3 are not) and `cohort_risk`, `sniper_density` are dropped (N/A) — their 0.25 weight is re-normalised onto the remaining sub-signals. The pillar keeps a `coverage` value and the response is flagged `coverage_caveat`, i.e. **standard vs advanced scores are not comparable**.

---

### Step 4 — Activity (token_overview only, max 100)

Four core sub-signals, all computed from Step 1 data, plus one optional pace signal. The weighting is **activity-first but risk-aware**: real trading activity still leads, but multi-interval momentum and multi-interval drawdown risk now carry more of the pillar than buy pressure. **Bot / wash-trade data is not available from the Birdeye API yet, so it is not scored** (`wash_trade_proxy` is informational only).

| sub-signal | weight | source |
|---|---|---|
| `engagement` | **0.40** | trade24h, v24hUSD, liquidity, marketCap, uniqueWallet24h |
| `buy_pressure` | 0.10 | per-frame vBuy/vSell + money flow |
| `momentum` | **0.30** | price / volume / wallet / liquidity changes across `MOMENTUM_FRAMES` |
| `drawdown` | **0.20** | nested-window give-back + 1h sell-off (see §4.4) |
| `pace` | (0.10) | **OFF by default** — enabled with `--pace` |

#### 4.1 Engagement — real trading activity first

| metric | ramp lo → hi |
|---|---|
| `trade24h` | log ramp `20 → 15,000` |
| `v24hUSD / liquidity` | `0.01× → 3.0×` |
| `v24hUSD / marketCap` | 0.75 % → 162 % turnover |
| `uniqueWallet24h` | 3 → 353 wallets |

`engagement = 100 × mean(available ramps)`.

**Ramp calibration.** The current implementation mixes two percentile-based anchors from the filtered `ALIVE` cohort in `new_solana.csv` (`liquidity >= USD 1,000`, `marketCap / liquidity <= 10`, `volume_30d_usd > 0`, plus trimming `volume_24h_usd < USD 1` and `liquidity < USD 100`, `n = 18,101`) with two practical live-trading anchors that behave better for early Solana launches.

| metric | cohort n | lo | hi | note |
|---|---|---|---|---|
| `trade24h` | — | 20 | 15,000 | practical log anchor for |

| | | | | |
|---|---|---|---|---|
| | | | | real tape activity |
| `v24hUSD / liquidity` | — | 0.01× | 3.0× | practical depth-vs-turnover anchor |
| `v24hUSD / marketCap` | 18,101 | 0.751 % | 162.02 % | filtered alive cohort |
| `uniqueWallet24h` | 17,995 | 3 | 353 | filtered alive cohort |

#### 4.2 Buy pressure — lighter confirmation layer

```
bp(frame)     = vBuy_USD(frame) / (vBuy_USD(frame) + vSell_USD(frame))
bp_score      = 100 × clamp01((bp - 0.43) / 0.24)      # 0.43 = neutral-ish, 0.67 = full
press         = Σ w_i · bp_score_i / Σ w_i
```

| frame | weight |
|---|---|
| 5m | 0.15 |
| 1h | 0.25 |
| 4h | 0.25 |
| 24h | 0.35 |

If 24h money flow is present, it is mixed in as a 10 % tilt: `score = 0.90 × press + 0.10 × (100 × ramp_s(mf_net_24h / v24hUSD, -10 %, +15 %))`.

#### 4.3 Momentum — secondary confirmation from multi-frame change

For each frame, the model blends up to four change inputs:

```
frame_tone =
    0.30 × tanh(price_change_pct / 35)
  + 0.35 × tanh(volume_change_pct / 90)
  + 0.25 × tanh(wallet_change_pct / 60)
  + 0.10 × tanh(liquidity_change_pct / 40)

momentum = 50 + 50 × Σ w_i · frame_tone_i / Σ w_i
```

`MOMENTUM_FRAMES` now spans four nested windows so a fading trend cannot hide in a single interval:

| frame | weight |
|---|---|

| | |
|---|---|
| 15m | 0.10 |
| 1h | 0.20 |
| 4h | 0.30 |
| 24h | 0.40 |

#### 4.4 Drawdown — nested-window give-back risk

A token that pumped over a long window but has since faded is a risk even while its point-in-time price is still up, so `drawdown` reads *give-back* between nested windows (only `token_overview` price-change frames are needed — no OHLC call):

```
for (long, short) in [(24h,1h), (24h,4h), (4h,1h)]:
    giveback = max(0, pc[long] - pc[short])          # % of the longer-window gain handed back
    risk_g   = smoothstep(giveback, 12 %, 65 %)
risk_s   = smoothstep(-pc[1h], 15 %, 60 %)           # live 1h sell-off
worst    = max(all risk terms)
drawdown = 100 × (1 - worst)
```

---

### Step 5 — Liquidity risk (advanced, Solana)

**Endpoint:** `/defi/v3/liquidity/ohlc/token` · **30 CU** · resolution `1m/5m/15m/30m/1h/4h/1D`, `count ≤ 100`. Default `15m × 96 candles = last 24 h` (`--liq-resolution`, `--liq-candles`).

Per candle: `close_liquidity_usd`, `close_exit_liquidity_usd`, `total_pairs`, `unix_time`.

#### 5.1 Statistics collected

The OHLC window is summarised into a nested `liquidity_stats` block — one `window` descriptor plus a full stat block for each series (`liq` = total liquidity, `exit` = exit liquidity):

```
window = { resolution, candles, span_hours, source }
liquidity / exit_liquidity = {
  current, first, min, max, median, mean,
  std, std_pct_of_mean,
  drawdown_pct, drawdown_from_peak_pct,
  trend_pct, candles
}
```

```
drawdown_pct             = 100 × (max - min) / max              # in-window swing
drawdown_from_peak_pct   = 100 × (max - current) / max          # below-window-peak now
trend_pct                = 100 × (current - first) / first      # net move over window
std_pct_of_mean          = 100 × std / mean                     # dispersion
```

With the default `15m × 96` window each series holds **96 candles = 24 h** (`span_hours = 24.0`), so min / max / median / drawdown are all 24 h statistics.

#### 5.2 Sub-signals

| sub-signal | weight | rule |
|---|---|---|
| `depth` | 0.30 | `0.45 × log_ramp_s(liquidity, 1 500, 800 000) + 0.55 × log_ramp_s(exit_liquidity, 1 000, 400 000)` — **exit liquidity weighted higher** (it is the realistic sell-side depth) |
| `stability` | 0.25 | `0.35 × (1 - ramp_s(drawdown_liquidity, 10 %, 60 %)) + 0.40 × (1 - ramp_s(drawdown_exit, 10 %, 60 %)) + 0.25 × ramp_s(exit_floor, 20 %, 90 %)` (needs ≥ 8 candles) |
| `valuation` | 0.20 | `0.45 × (1 - log_ramp_s(mcap/liquidity, 20, 500)) + 0.55 × (1 - log_ramp_s(mcap/exit_liquidity, 40, 1000))` |
| `trend` | 0.15 | `100 × (0.5 + 0.5 × tanh(Δliquidity / 0.30))` over the window |
| `pairs` | 0.10 | `100 × ramp_s(max(total_` |

| | | |
|---|---|---|
| | | `pairs), 1, 5)` |

**Why log-spaced.** Solana liquidity spans 3+ orders of magnitude — from tokens with a few dollars of pooled depth to blue chips with millions. A linear ramp reaches full marks after about `USD 30k` and then every token in the `USD 20k` - `USD 1M` band scores the same, so `USD 20k` looks "high" when it is really mid-range. The log ramp spreads score evenly across 1.5 orders per decade, so `USD 20k` lands around 35–40/100 and the ceiling is only reached near `USD 800k` liquidity / `USD 400k` exit liquidity.

The `USD 2,000` **line is a ramp start, not a cliff**: `log_ramp_s(exit, 1000, 400000)` is still low at `USD 2,000` (≈0.04) and rises smoothly from there, so the depth sub-signal is heavily penalised at `USD 2k` without being pinned to a fixed 0.

#### 5.3 The liquidity-drawdown gate (Step 6 input)

The gate reads **deterioration, not depth**. Absolute depth is already scored by the `depth` sub-signal above, so gating on it as well penalised one weakness twice — a token with `USD 5k` exit liquidity scored low on `depth` *and* then lost a further ~25 % of its **total** score. The gate therefore only reads how far liquidity has fallen from its recent peak over the OHLC window:

```
liq_pen  = 0.10 × smoothstep(ramp(liquidity_drawdown_from_peak_pct / 100, 0.20, 0.70))
exit_pen = 0.15 × smoothstep(ramp(exit_liquidity_drawdown_from_peak_pct / 100, 0.20, 0.70))
pen      = max(liq_pen, exit_pen)          # worst term binds — not summed
gate     = 1 - pen                         # floor 0.85 → at most a 15 % total discount
```

| drawdown from peak | `liq_pen` | `exit_pen` | gate |
|---|---|---|---|
| ≤ 20 % | 0.000 | 0.000 | 1.000 |
| 35 % | 0.014 | 0.021 | 0.979 |
| 50 % | 0.040 | 0.060 | 0.940 |
| 70 %+ | 0.100 | 0.150 | 0.850 |

Three things follow:

**Worked example (BP token).** Liquidity `drawdown_from_peak = 45 %`, exit `drawdown_from_peak = 30 %`:

```
liq_pen  = 0.10 × smoothstep(ramp(0.45, 0.20, 0.70)) = 0.10 × 0.500 = 0.050
exit_pen = 0.15 × smoothstep(ramp(0.30, 0.20, 0.70)) = 0.15 × 0.104 = 0.016
```

```
pen      = max(0.050, 0.016) = 0.050
gate     = 1 - 0.050 = 0.950   (binding = liquidity_drawdown)
```

`gate_detail` reports `liquidity_drawdown_pct`, `exit_drawdown_pct`, `liquidity_penalty`, `exit_penalty`, `binding` (`exit_drawdown` / `liquidity_drawdown` / `none`) and a reserved `holder_change_penalty` (always `null` — multi-interval holder change is a future deterioration input).

Depth: `log_ramp_s(7.4 M, 1 500, 800 000) = 1` → 100; `log_ramp_s(4.0 M, 1 000, 400 000) = 1` → 100 ⇒ `depth = 100`. Valuation: `mcap/liquidity = 281.05 / 7.4 = 37.98` → `1 - smooth(0.199) = 0.897` → 89.7; `mcap/exit = 70.26` → `1 - smooth(0.175) = 0.919` → 91.9 ⇒ `valuation = 0.45 × 89.7 + 0.55 × 91.9 = 90.9`.

#### 5.4 Standard mode

No OHLC call ⇒ `stability` and `trend` are N/A (0.40 of the pillar weight) and `exit_now` is unknown ⇒ `depth` uses **total liquidity only** (single log ramp `USD 1,500` → `USD 800,000`). With no OHLC window there is no drawdown either, so the **gate is neutral at `1.000`** — the Liquidity pillar's own `depth` / `valuation` sub-scores carry the whole depth judgement. This is a materially weaker safety net, hence the `coverage_caveat`.

---

### Step 6 — Smart money (advanced)

Three focus groups: **smart_degen / smart traders**, **whales**, **KOLs**. Mapped onto the Birdeye endpoints that actually exist (anything not available is dropped — *"what Birdeye has, we use; what it does not, we skip"*).

| group | source | what we read |
|---|---|---|
| Smart money / whales / KOL flow | `/defi/v3/token/money-flow` (10 CU) | `netUsd`, `totalTxBuy`, `totalTxSell`, `uniqTrader` for `smart_money`, `whales`, `kol` across 15m / 1h / 4h / 24h |
| Cohort supply + buy/sell history | `holder-profile.tags[...]` (Step 3) | `holder_count`, `percent_of_supply`, `buy_volume_usd`, `sell_volume_usd` |

| | | |
|---|---|---|
| Top holders (wallet mode) | `/defi/v3/token/holder?mode=wallet&get_holder_infos=true&limit=100` (30 CU) | wallet detail for renderer context and holder-side inspection |

#### 6.1 Sub-signals

| sub-signal | weight | formula |
|---|---|---|
| `sm_presence` | 0.25 | weighted mean of `ramp_s(smart_trader.holder_count, 0, 25)` and `ramp_s(smart_trader.percent_of_supply, 0, 8)` |
| `sm_flow` | 0.30 | `0.55 × cohort_flow(smart_money, -5 %, +12 %) + 0.45 × clamp01((buy_ratio - 0.40) / 0.35)` |
| `sm_quality` | 0.20 | `0.55 × ramp_s(smart_trader.percent_of_supply, 0, 10) + 0.45 × clamp01((hold_ratio - 0.45) / 0.30)` |
| `whale` | 0.15 | weighted mean of `ramp_s(whale_count, 0, 20)`, `ramp_s(whale_pct, 0, 12)`, `cohort_flow(whales, -8 %, +8 %)` |

| | | |
|---|---|---|
| `kol` | 0.10 | weighted mean of `ramp_s(kol_count, 0, 40)`, `ramp_s(kol_pct, 0, 8)`, `cohort_flow(kol, -5 %, +6 %)` |

#### 6.1.1 `cohort_flow` — multi-interval net-flow scoring

The whale / KOL / smart-money flow terms no longer read 24 h only. `cohort_flow` reads the full `(tag × frame)` money-flow grid exposed by `token_overview.moneyFlow` across `SM_FLOW_FRAMES`:

```
for (frame, w) in [(5m,0.15),(1h,0.25),(4h,0.25),(24h,0.35)]:
    pct(f)   = net_usd(tag, f) / volume_usd(f)
    score(f) = 100 × smoothstep(pct(f), lo, hi)
mean       = Σ w·score / Σ w
agree      = share of frame-weight agreeing in sign with mean
cohort_flow = 50 + (mean - 50) × (0.75 + 0.25 × agree)
```

Derived quantities:

```
hold_ratio = smart_buy_volume / (smart_buy_volume + smart_sell_volume)
buy_ratio  = smart_money_buy_txs / (smart_money_buy_txs +
smart_money_sell_txs)
```

**Bullish / bearish mapping**

| signal | bullish | bearish |
|---|---|---|
| smart-money flow | positive `net_flow_24h / vol`, buy tx ratio well above 0.40 | strongly negative net flow, sell-heavy tx ratio |
| smart-money quality | smart-trader cohort keeps a meaningful supply share and buy share | smart-trader cohort is tiny or already net-selling |
| whales | more whale wallets, more whale supply, positive whale net flow | no whale presence or persistent whale outflow |
| KOL | more KOL wallets, meaningful KOL supply | no KOL cohort or KOL outflow |

| | | |
|---|---|---|
| | share, positive KOL net flow | |

Any sub-signal with absent source data is marked `N/A`, dropped from the denominator, and excluded from the pillar coverage.

#### 6.2 Standard mode

Smart Money is **not measurable** in standard mode (no token money flow, no top holders, no holder-profile) ⇒ the pillar is removed from the weights and re-normalised across the other four. With the current `default` preset, the effective standard-mode weights become:

---

### Step 7 — Compose: weighted mean × gate

```
1. weights = preset (default / strict-liquidity / security-first)
2. standard  -> drop "smart_money", keep the rest
3. chain == sol -> weights += SOL_WEIGHT_SHIFT (security -0.06, holders
   +0.03, liquidity +0.03)
4. drop any pillar whose score is None
5. re-normalise the surviving weights to sum 1.0
6. base  = Σ norm_w(k) × pillar_score(k)
7. gate  = 1 - max(liq_pen, exit_pen)          (see Step 5.3 — drawdown
   only)
8. FINAL = base × gate
```

Weight re-normalisation is what makes the two modes both end on 0–100 despite using a different number of pillars. `score.weights_used` and `score.pillar_points` expose the normalised weights and each pillar's contribution in points, so the arithmetic is fully auditable. The declared profile *after* the Solana shift is echoed in `meta.weights_declared` and the applied delta in `meta.weights_shift` (`null` off-Solana).

**Worked example — advanced, preset `default`, chain `sol`:**

| pillar | score | weight (Solana, normalised) | points |
|---|---|---|---|
| Security | 37.1 | 0.17 | 6.3 |
| Holders | 40.8 | 0.28 | 11.4 |
| Activity | 51.6 | 0.20 | 10.3 |
| Liquidity | 90.5 | 0.25 | 22.6 |
| Smart Money | 53.4 | 0.10 | 5.3 |
| **base** | | | **56.0** |

| | | | |
|---|---|---|---|
| gate | | | × 0.950 (liquidity drawdown 45 %) |
| **FINAL** | | | **53.2 → rating D** |

The same pillar scores on a **non-Solana** chain use the unshifted `default` profile (0.23 / 0.25 / 0.20 / 0.22 / 0.10) → `base = 54.3`. In `standard` mode the Smart Money weight is dropped and the rest re-normalise to 1.0 ( `default`, non-Solana → 0.256 / 0.278 / 0.222 / 0.244; `default`, Solana → 0.189 / 0.311 / 0.222 / 0.278).

---

### Step 8 — Rating & rendering

| Rating | Score | Verdict |
|---|---|---|
| A+ | ≥ 95 | 🔥 HOT |
| A | ≥ 85 | 🟢 STRONG |
| B+ | ≥ 80 | 🟢 OK |
| B | ≥ 70 | 🟢 FAIR |
| C+ | ≥ 65 | 🟡 WATCH |
| C | ≥ 55 | 🟡 CAUTION |
| D | ≥ 40 | 🟠 RISKY |
| E | 20 – <40 | 🔴 HIGH RISK |
| F | < 20 | ⛔ CRITICAL |

The verdict is a **risk label, not a trade call** — the skill does not emit buy/sell advice.

Hard blocks force `final = 0` / rating `F`.

**Render** ( `--format` ): `ui` (structured payload for the web UI), `json` / `raw-json` (machine readable, includes `inputs` for re-score), `telegram` (compact), `plain` (terminal). All renders print the mandatory `⚡CU cost` footer from `meta.cu_billed_total`.

### Cache & history

```
~/.cache/birdeye-skills-runs/
  run_<ts>_token-dd-score.raw.json     # full result
  run_<ts>_token-dd-score.input.json   # raw endpoint inputs -> enables --rescore
```

`--rescore <run_id|path|base64>` replays the stored inputs and re-runs Steps 1–8 locally: **0 API calls, 0 CU**. Use it to switch `--weights-preset`, turn on `--pace`, widen `--top-n`, or toggle `--explain` without paying for the data twice. The re-scored response adds the caveat `0-CU re-score from cache`.

---

### CU breakdown mapping

`meta.cu_breakdown` uses the 5 canonical keys:

| key | endpoints | standard | advanced |
|---|---|---|---|
| `base` | token_overview (21) + token_security (30) | 51 | 51 |
| `list` | distribution (30) + holder-profile (30) + top_holders (30) | 0 | 90 |
| `advanced_security` | token_money_flow (10) + liquidity OHLC (30) | 0 | 40 |
| `risk_history` | reserved | 0 | 0 |
| `refunded_rollback` | CU returned for Solana-only endpoints skipped on other chains | 0 | 0 |
| `cu_billed_total` | | **51** | **181** |

---

### Caveats